

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I P. Akhila/ 192472320 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Akhila

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Aim

To study the impact of image preprocessing techniques such as noise removal and normalization on image quality and feature detection, and to analyze how preprocessing improves the accuracy and reliability of feature extraction.

Algorithm

1. Read the input image using OpenCV.
2. Apply a Gaussian Filter to remove noise from the image.
3. Normalize the filtered image to improve intensity distribution.
4. Convert both the original and preprocessed images into grayscale.
5. Apply Canny Edge Detection on both images.
6. Compare the detected edges before and after preprocessing.
7. Analyze the improvement in image quality and feature detection accuracy.
8. Display and save the output images.

Theoretical Concept

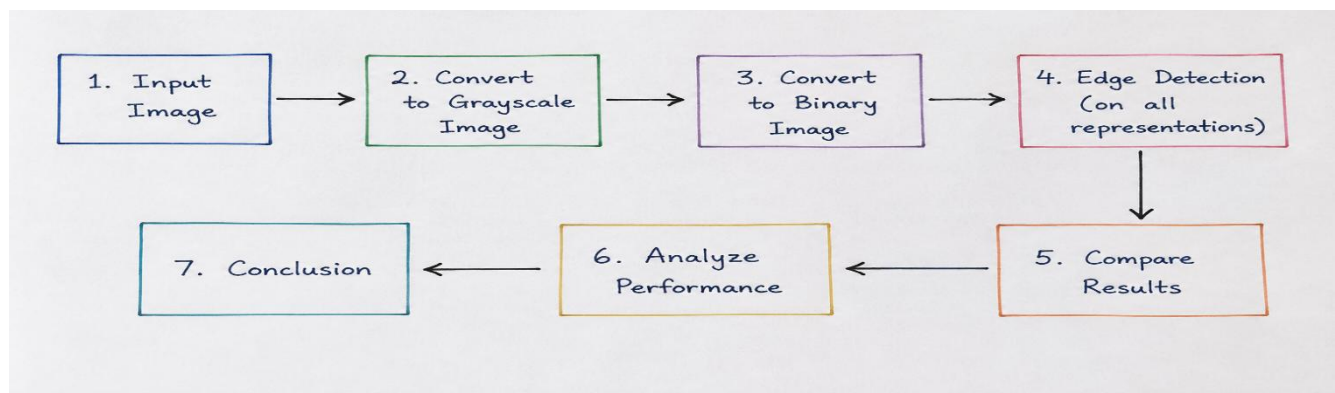
- **Image preprocessing** is the first step in computer vision that improves image quality before feature detection and analysis.
- **Noise removal** eliminates unwanted pixel variations using filters such as the **Gaussian Filter**, resulting in a smoother and clearer image.
- **Normalization** adjusts the pixel intensity values to a common range, improving brightness and contrast for consistent image representation.
- Preprocessing helps **edge detection and feature extraction algorithms** identify important image features more accurately by reducing noise and enhancing clarity.

- These preprocessing techniques improve the **accuracy, reliability, and performance** of computer vision applications such as object detection, image matching, and image classification.

Justification for Selecting the Techniques

- The Gaussian Filter was selected because it effectively reduces random noise without significantly affecting the important structures of the image.
- Normalization was chosen because it provides consistent intensity values across the image, making feature detection algorithms more reliable.
- The combination of these two techniques improves image quality and prepares the image for accurate feature extraction.

Experimental Design/ Workflow



Code

```

import cv2

import matplotlib.pyplot as plt

image = cv2.imread("Picture4.jpg")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

gaussian = cv2.GaussianBlur(image, (5, 5), 0)

normalized = cv2.normalize(gaussian, None, 0, 255, cv2.NORM_MINMAX)

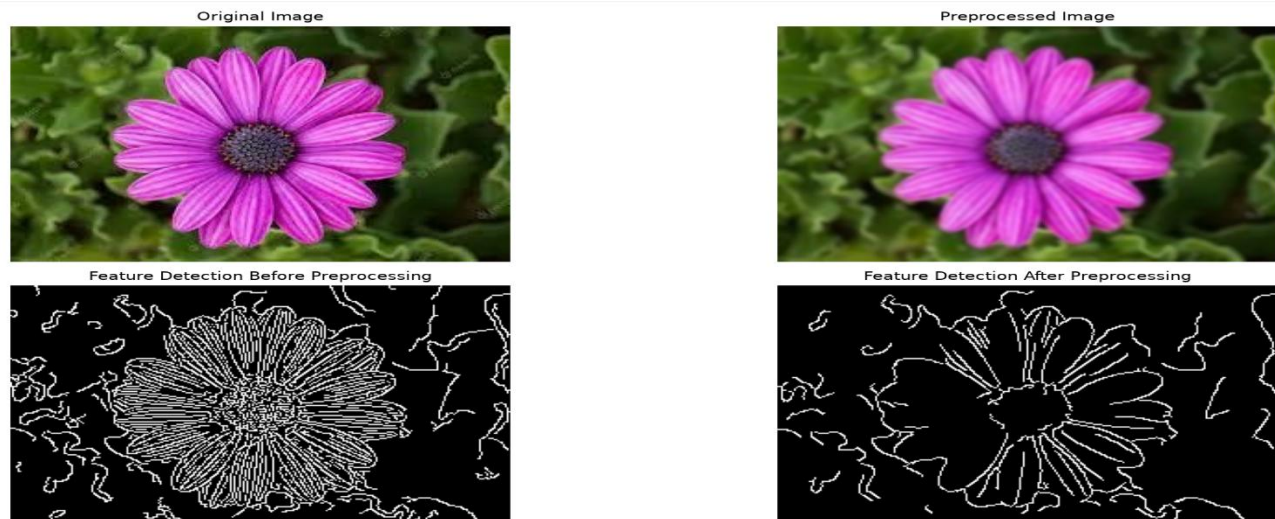
gray_original = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

gray_processed = cv2.cvtColor(normalized, cv2.COLOR_RGB2GRAY)

```

```
edges_original = cv2.Canny(gray_original, 100, 200)
edges_processed = cv2.Canny(gray_processed, 100, 200)
plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2,2,2)
plt.imshow(normalized)
plt.title("Preprocessed Image")
plt.axis("off")
plt.subplot(2,2,3)
plt.imshow(edges_original, cmap="gray")
plt.title("Feature Detection Before Preprocessing")
plt.axis("off")
plt.subplot(2,2,4)
plt.imshow(edges_processed, cmap="gray")
plt.title("Feature Detection After Preprocessing")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output



Result

The preprocessing techniques successfully improved the image quality by reducing noise and normalizing the intensity values. As a result, feature detection produced clearer and more accurate edges, making the image more suitable for further computer vision analysis.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Aim

To study different image representations by converting a color image into grayscale and binary images, and to analyze their impact on edge detection and feature extraction in computer vision.

Algorithm

1. Read the input image using OpenCV.
2. Convert the RGB image into a grayscale image.
3. Convert the grayscale image into a binary image using thresholding.
4. Apply Canny edge detection to the RGB, grayscale, and binary images.

5. Compare the outputs obtained from each image representation.
6. Analyze the effect of each representation on feature extraction.
7. Display and save all output images.

Theoretical Concept

- Image representation is the process of representing an image in different formats for efficient processing and analysis.
- A grayscale image contains only intensity values, reducing computational complexity while preserving important information.
- A binary image consists of only two pixel values (0 and 255), making object separation and segmentation easier.
- Different image representations influence the performance of edge detection and feature extraction algorithms.
- Selecting the appropriate image representation improves the accuracy and efficiency of computer vision applications.

Justification of the Selected Technique

- Grayscale conversion reduces computational cost while preserving essential image information.
- Binary conversion simplifies the image by separating objects from the background.
- Comparing different image representations helps identify the most suitable format for edge detection and feature extraction.
- OpenCV provides efficient functions for image conversion and analysis.

Experimental Design/ Workflow

Code

```
import cv2  
  
import matplotlib.pyplot as plt  
  
image = cv2.imread("Picture4.jpg")
```

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
edge_rgb = cv2.Canny(cv2.cvtColor(image, cv2.COLOR_RGB2GRAY),100,200)
edge_gray = cv2.Canny(gray,100,200)
edge_binary = cv2.Canny(binary,100,200)
plt.figure(figsize=(10,8))
plt.subplot(2,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2,3,2)
plt.imshow(gray,cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.subplot(2,3,3)
plt.imshow(binary,cmap="gray")
plt.title("Binary Image")
plt.axis("off")
plt.subplot(2,3,4)
plt.imshow(edge_rgb,cmap="gray")
plt.title("Edges (Original)")
plt.axis("off")
plt.subplot(2,3,5)
```

```
plt.imshow(edge_gray,cmap="gray")  
plt.title("Edges (Grayscale)")  
plt.axis("off")  
plt.subplot(2,3,6)  
plt.imshow(edge_binary,cmap="gray")  
plt.title("Edges (Binary)")  
plt.axis("off")  
plt.tight_layout()  
plt.show()
```

Output

Original Image



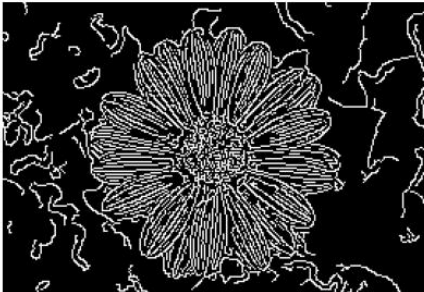
Grayscale Image



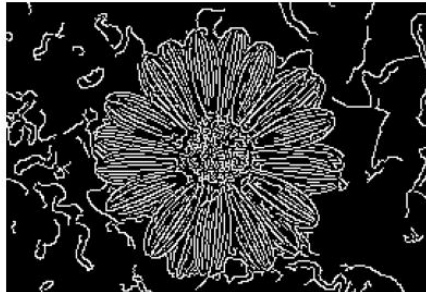
Binary Image



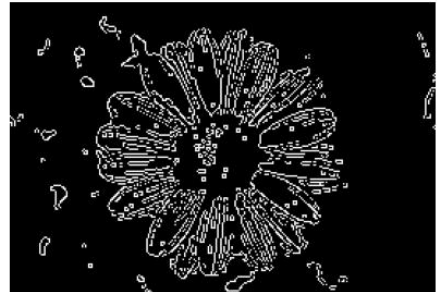
Edges (Original)



Edges (Grayscale)



Edges (Binary)



Result

The image was successfully converted into grayscale and binary representations. The grayscale image produced better edge detection and feature extraction results, while the binary image was more effective for object separation.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Aim

To compare the performance of Sobel and Canny edge detection techniques on the same input image and analyze their effectiveness for different image conditions.

Algorithm

1. Read the input image using OpenCV.
2. Convert the image into grayscale.
3. Apply the Sobel edge detection technique.
4. Apply the Canny edge detection technique.
5. Compare the detected edges produced by both methods.
6. Analyze the quality and accuracy of the detected edges.
7. Display and save the output images.

Theoretical Concept

- Edge detection is used to identify object boundaries by detecting sudden changes in pixel intensity.
- The Sobel operator calculates image gradients in the horizontal and vertical directions to detect edges.

- The Canny edge detector uses Gaussian filtering, gradient calculation, non-maximum suppression, and double thresholding to detect accurate edges.
- Sobel is simple and computationally efficient, whereas Canny provides more precise and continuous edge detection.
- Edge detection is widely used in image segmentation, object detection, medical imaging, and computer vision applications.

Justification of the Selected Technique

- Sobel is selected because it is simple, fast, and effective for detecting basic edges.
- Canny is selected because it produces more accurate and less noisy edge detection.
- Comparing both techniques helps identify the most suitable method for different image processing applications.
- OpenCV provides efficient functions to implement both edge detection techniques.

Code

```
import cv2

import matplotlib.pyplot as plt

image = cv2.imread("Picture4.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel = cv2.addWeighted(cv2.convertScaleAbs(sobelx), 0.5,
                        cv2.convertScaleAbs(sobely), 0.5, 0)

canny = cv2.Canny(gray, 100, 200)

plt.figure(figsize=(10,4))

plt.subplot(1,3,1)

plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original Image")
plt.axis("off")
plt.subplot(1,3,2)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel Edge Detection")
plt.axis("off")
plt.subplot(1,3,3)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Output



Result

Both edge detection techniques were successfully implemented. The Canny method produced clearer and more accurate edges than the Sobel method, making it more suitable for feature extraction and object detection.